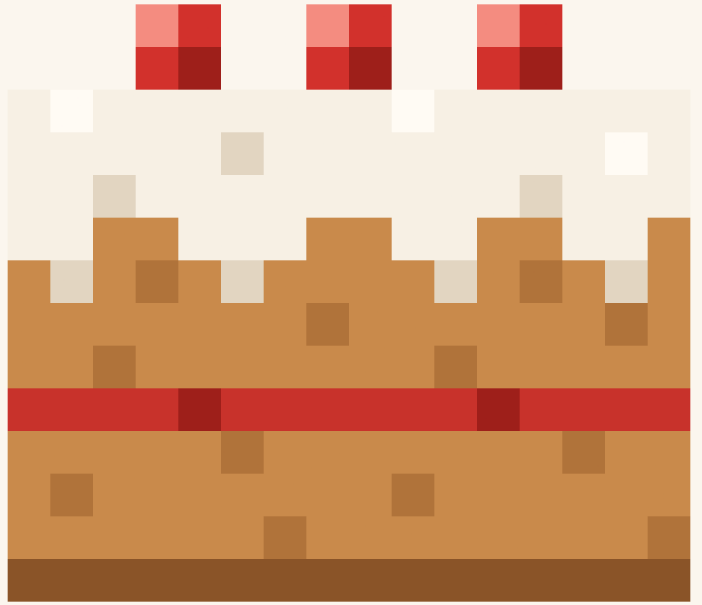
layer cake.

Your devices, one model.



A little from each device, more possible together. Layer Cake splits a language model's layers across the computers you already own.

HackOHIO 2026 · Aarush Maddula · Connor Cochran · David Stephenson · Yash Joshi

THE PROBLEM

Good models outgrow the laptops we own.

19.9 GB

Weights of Qwen2.5-32B at Q4_K_M, before context and scratch memory.

›

16 GiB

Physical RAM in each of the two Macs we had, and the owner still needs some of it.

Cloud GPUs fix the size but cost money and send your prompts off-device. We wanted the machines already in the room to pool their memory instead.

Source: experiments/model-validation/SHARD_RUNNER.md

THE IDEA

Every device has a part.



layer 0 layer 63

M1 Air · layers 0–31

Loads only its half of the weights and turns tokens into hidden states.

20 KB / token

over Iroh

M2 Pro · layers 32–63

Finishes the pass, picks the next token and streams the reply.

ARCHITECTURE

One leader, many workers, two backends.

CLIENT

Dashboard, chat or CLI

Shows devices, the layer map and queued, running and finished jobs.

LEADER

Job API, queue, history

llama: llama-server over ggml RPC bridges, from a placement plan

shard: in-process front end that dials layer-range workers

WORKERS

worker: supervises rpc-server and bridges it to the leader

shard-serve: runs one layer range per session

Iroh mesh underneath: QUIC with NAT traversal and encryption. Workers serve only node IDs on their allowlist.

HOW A JOB RUNS

From prompt to streamed reply, one token at a time.

01

Queue

The job API takes a chat request off its queue and renders it with the model's chat template.

02

Dial

The leader dials the prefix worker by node ID and starts its own final-stage runner.

03

Pass

Token IDs go out, hidden states come back, the final stage picks the next token.

04

Stream

Each token loops back to the prefix; text streams to the browser as it is made.

Each worker runs one session at a time with a fresh runner, so jobs never share state on a device.

THE RESULT · OCTOBER 24, 2026

A 32B model ran across two 16 GiB Macs.

Qwen2.5-32B Q4_K_M, split 32/32 over Iroh. Neither Mac ever loaded the full set of weights.

> *“Splitting a language model across two computers distributes the computational load, allowing each machine to handle a portion of the model.”*
>
> The model’s own reply, 41 output steps on an M1 Air + M2 Pro

WHAT WE MEASURED

Each Mac carried about half the model.

Metric	M1 prefix	M2 suffix
Owned layers	0–31	32–63
Runtime buffers	9.96 GB	10.18 GB
Peak task footprint	10.13 GB	10.97 GB
Median decode compute	171 ms	65 ms

2.33

output steps per second after prefill

20 KB

activation per step, against a 24-byte token request

One cold run over a relayed Iroh path; not a controlled throughput comparison. Source: SHARD_RUNNER.md

PLACEMENT

Measured profiles become a launch plan.

Profiles

Memory budget, compute cost and link cost for each device.

Planner

Searches device orders and layer ranges for the fastest decode that fits.

Plan

Layer slices, device mappings and llama.cpp launch arguments.

Before launch: the leader checks coverage, budgets and topology, and refuses a bad plan.

After a run: supplied evidence is checked, so a healthy endpoint is never mistaken for proof.

HISTORY SYNC AND FAILOVER

If the leader drops, the conversation survives.

Every device keeps a copy

After each job the leader syncs the history to all workers, who save it to disk.

Survivors agree on one

Newest history wins, then longest connected, then lowest node ID. Same inputs, same winner.

A new epoch begins

The winner takes over one epoch higher, so a returning old leader steps down.

⚠ Working in a simulator and 31 tests; not yet wired into live nodes.

WHERE IT STANDS

Working pieces, honest gaps.

Working

- Iroh mesh, RPC bridge and peer allowlists
- Budget-aware placement with preflight checks
- Leader and worker launch with heartbeats
- Shard backend and the Layer Cake dashboard
- Builds for Windows, Linux and macOS

Next

- Emoji-code pairing, wired to the core
- Wire history sync into live nodes
- Let a worker host a shard the leader assigns
- Enforce hard per-device memory caps
- More than two devices in one run

TRY IT

Better with company.

On the device that holds layers 0–31

```
llm-cluster shard-serve --model prefix-0-32.gguf --range 0:32 \
--runner mesh-shard-runner --allow-peer <leader node id>
```

On the leader, then open the dashboard with ./mesh-ui

```
llm-cluster leader --backend shard --shard-manifest m.json
```

layer cake. Thank you.

Aarush Maddula · Connor Cochran · David Stephenson · Yash Joshi